CHAPTER 1
ASP.NET Core

CONTENT

- Putting ASP.NET Core in Context
- Understanding the Application Frameworks
- Choosing a Code Editor
- Creating an ASP.NET Core Project
- Running the ASP.NET Core Application
- Understanding Endpoints

PUTTING ASP.NET CORE IN CONTEXT

Putting ASP.NET Core in Context

- ASP.NET Core is Microsoft's web development platform. The original ASP.NET was introduced in 2002, and it has been through several reinventions and reincarnations to become ASP.NET Core 8
- ASP.NET Core consists of a platform for processing HTTP requests, a series of principal frameworks for creating applications, and secondary utility frameworks that provide supporting features

Putting ASP.NET Core in Context

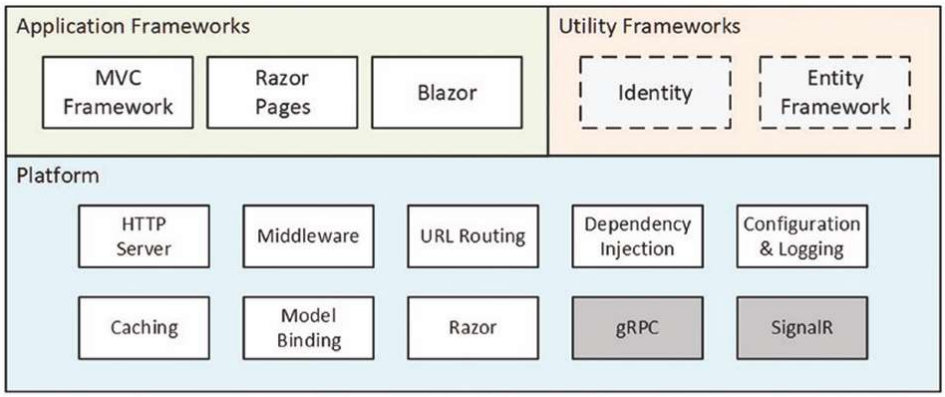


Figure 1-1. *The structure of ASP.NET Core*

UNDERSTANDING THE APPLICATION FRAMEWORKS

The Application Frameworks

- Understanding the MVC Framework:
  - The MVC Framework was introduced in the early ASP.NET, **long** before .NET Core and .NET 6 were introduced
  - MVC stands for **Model-View-Controller**, which is a design pattern that describes the shape of an application. The MVC pattern emphasizes **separation of concerns**, where areas of functionality are defined **independently**, which was an effective antidote to the indistinct architectures that Web Pages led to
  - The MVC Framework remains an important part of ASP.NET Core, but the way it is commonly used has changed with the rise of **single-page applications (SPAs)**.

The Application Frameworks

- Understanding Razor Pages:
  - One drawback of the MVC Framework is that it can **require a lot of preparatory work** before an application can start producing content. Despite its structural problems, one advantage of Web Pages was that simple applications could be created in a couple of hours
  - Razor Pages takes the development ethos of Web Pages and implements it using the platform features originally developed for the MVC Framework. Code and content are mixed to form self-contained pages; this re-creates the speed of Web Pages development without some of the underlying technical problems

The Application Frameworks

- Understanding the Utility Frameworks:
  - Two frameworks are closely associated with ASP.NET Core but are not used directly to generate HTML content or data.
  - **Entity Framework Core** is Microsoft's object-relational mapping (ORM) framework, which represents data stored in a relational database as .NET objects. Entity Framework Core can be used in any .NET application, and it is commonly used to access databases in ASP.NET Core applications.
  - **ASP.NET Core Identity** is Microsoft's authentication and authorization framework, and it is used to validate user credentials in ASP.NET Core applications and restrict access to application features.

The Application Frameworks

- Understanding the ASP.NET Core Platform:
  - The ASP.NET Core platform contains the low-level features required to receive and process HTTP requests and create responses. There is an integrated HTTP server, a system of middleware components to handle requests, and core features that the application frameworks depend on, such as URL routing and the Razor view engine.

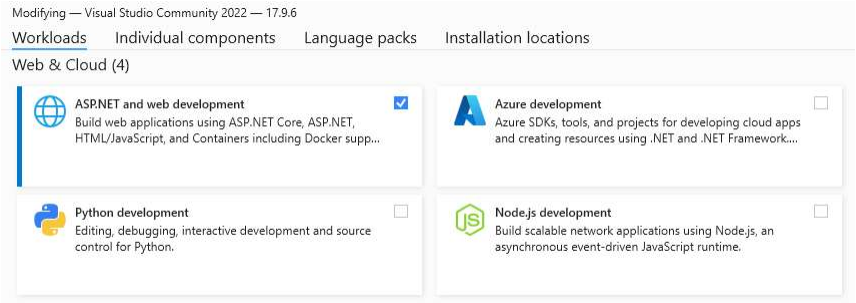
CHOOSING A CODE EDITOR

Choosing a Code Editor

- 2 common Code Editor for ASP.NET Core:
  - Microsoft provides a choice of tools for ASP.NET Core development: **Visual Studio** and **Visual Studio Code**. Visual Studio is the traditional development environment for .NET applications, and it offers an enormous range of tools and features for developing all sorts of applications. But it can be resource-hungry and slow, and some of the features are so determined to be helpful they get in the way of development.
  - Visual Studio Code is a lightweight alternative that doesn't have the bells and whistles of Visual Studio but is perfectly capable of handling ASP.NET Core development.

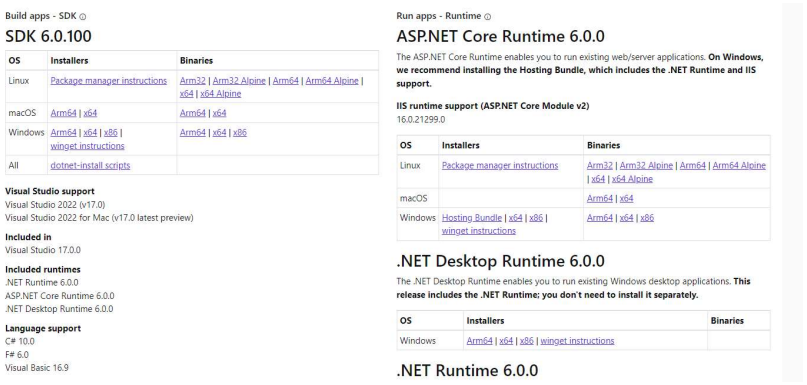
Choosing a Code Editor

- Install Visual Studio Community:
  - Link: https://visualstudio.microsoft.com/vs/community/
  - Ensure that the **"ASP.NET and web development"** workload is checked



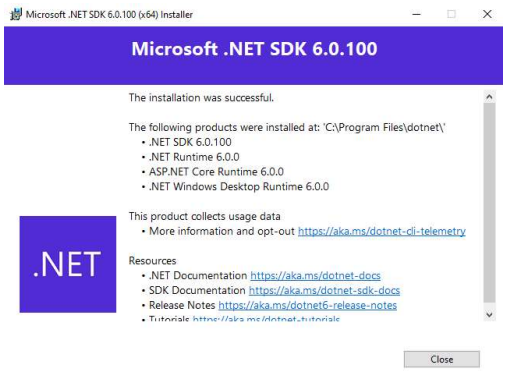
Choosing a Code Editor

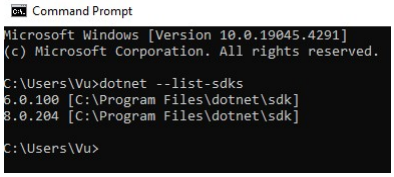
- Installing the .NET SDK:
  - Go to https://dotnet.microsoft.com/en-us/download/dotnet/6.0 and download the installer for version 6.0.0 of the .NET SDK



Choosing a Code Editor

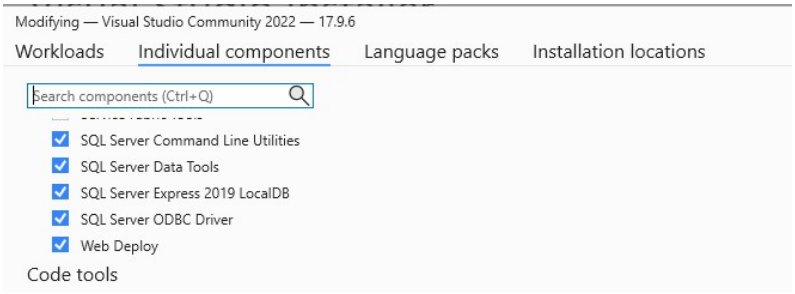
- After installed, open cmd and run: **dotnet --list-sdks**





Choosing a Code Editor

- Install Visual Studio Community:
  - Select the “Individual components” section at the top of the window and ensure the **SQL Server Express 2019 LocalDB option is checked**



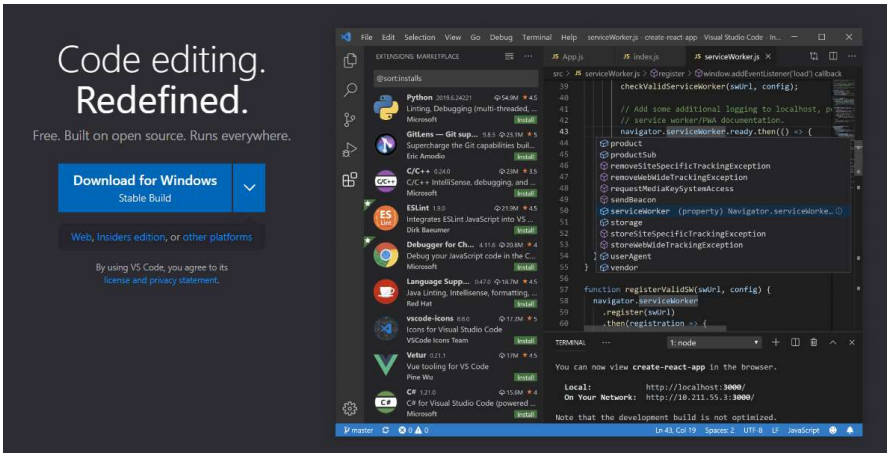
Choosing a Code Editor

- Install MS SQL SERVER:

Microsoft®
SQL Server®
2022

Choosing a Code Editor

- Install VSCode: https://code.visualstudio.com/



CREATING AN ASP.NET CORE PROJECT

Creating an ASP.NET Core Project

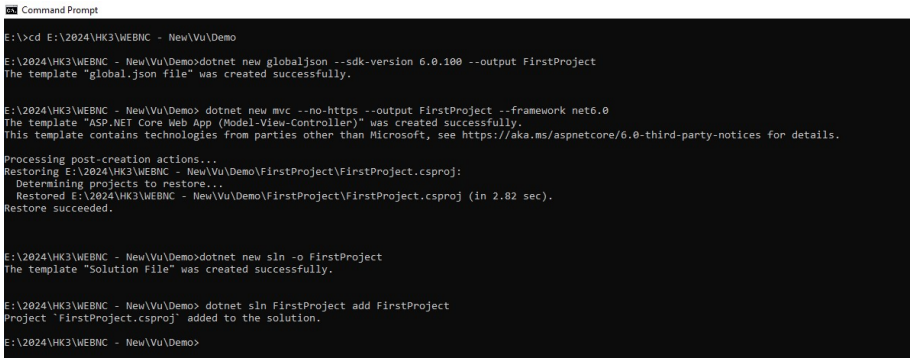
- Use command line:
  - The most direct way to create a project is to use the command line. Open a new PowerShell command prompt from the Windows Start menu, navigate to the folder where you want to create your ASP.NET Core projects, and run the commands

Creating an ASP.NET Core Project

- Use command line:
  - dotnet new globaljson --sdk-version 6.0.100 --output FirstProject
  - dotnet new mvc --no-https --output FirstProject --framework net6.0
  - dotnet new sln -o FirstProject
  - dotnet sln FirstProject add FirstProject

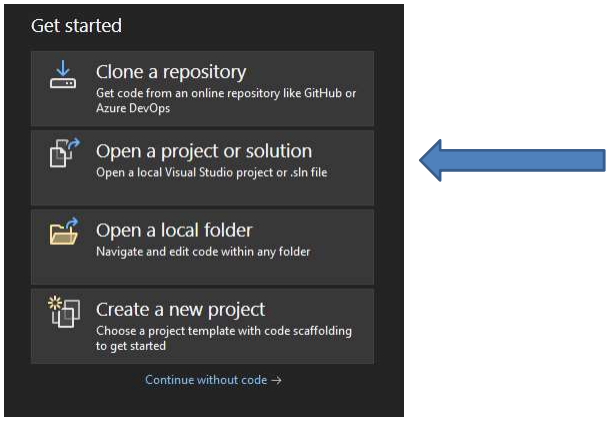
Creating an ASP.NET Core Project

- Use command line: Result



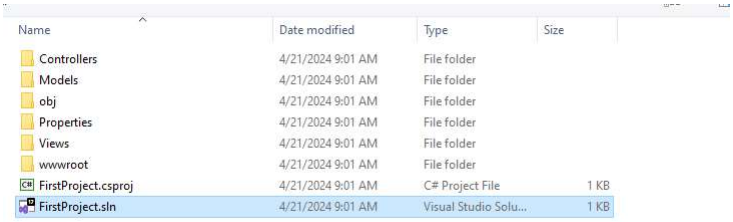
Creating an ASP.NET Core Project

- Opening the Project Using Visual Studio:



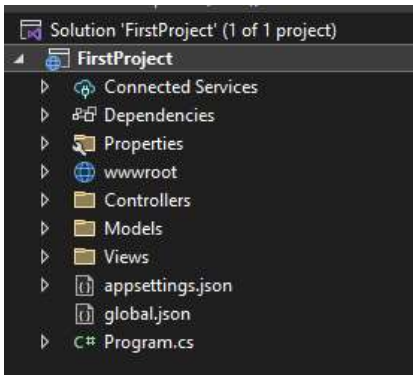
Creating an ASP.NET Core Project

- Opening the Project Using Visual Studio: choose the **sln** file



Creating an ASP.NET Core Project

- Opening the Project Using Visual Studio: choose the **sln** file



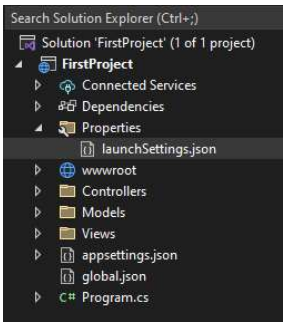
RUNNING THE ASP.NET CORE APPLICATION

Running the ASP.NET Core Application

- When the project is created, a file named **launchSettings.json** is created in the Properties folder, and it is this file that determines which HTTP port ASP.NET Core will use to listen for HTTP requests.
- Open this file in your chosen editor and change the ports in the URLs it contains to **5000**

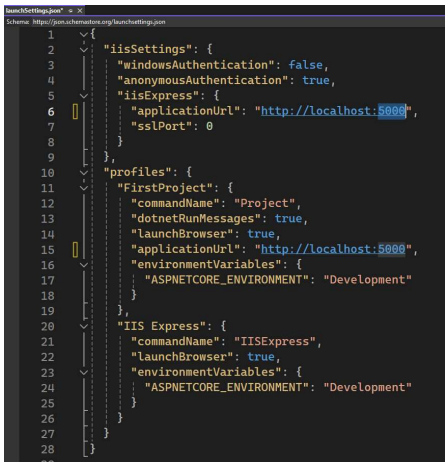
Running the ASP.NET Core Application

- When the project is created, a file named **launchSettings.json** is created in the Properties folder, and it is this file that determines which HTTP port ASP.NET Core will use to listen for HTTP requests.
- Open this file in your chosen editor and change the ports in the URLs it contains to **5000**



Running the ASP.NET Core Application

- Open this file in your chosen editor and change the ports in the URLs it contains to **5000**



Running the ASP.NET Core Application

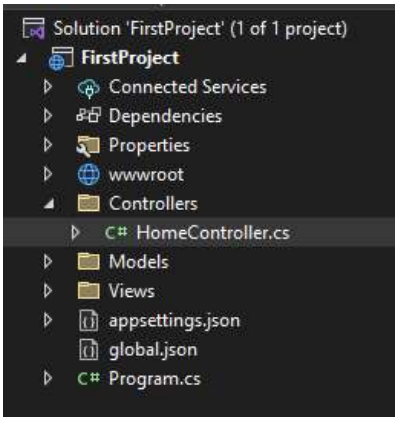
- Press F5 to run the program

UNDERSTANDING ENDPOINTS

Understanding Endpoints

- In an ASP.NET Core application, incoming requests are handled by **endpoints**.
- The endpoint that produced the response in last section is an **action**, which is a **method** that is written in C#.
- An action is defined in a **controller**, which is a **C# class** that is derived from the **Microsoft.AspNetCore.Mvc.Controller** class, the built-in controller base class.
- The convention in ASP.NET Core projects is to put controller classes in a **folder** named **Controllers**, which was created by the template used to set up the project

Understanding Endpoints

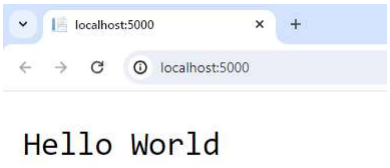


Understanding Endpoints

```
3 references
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    0 references
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    0 references
    public IActionResult Index()
    {
        return View();
    }

    0 references
    public IActionResult Privacy()
    {
        return View();
    }

    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
    0 references
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }
}
```

Understanding Endpoints

```
0 references
public string Index()
{
    return "Hello World";
}
```



UNDERSTANDING ROUTES

Understanding Routes

- The ASP.NET Core routing system is responsible for selecting the endpoint that will handle an HTTP request.
- A route is a rule that is used to decide how a request is handled. When the project was created, a default rule was created to get started. You can request any of the following URLs, and they will be dispatched to the Index action defined by the Home controller:
  - /
  - /Home
  - /Home/Index

Understanding Routes

- So, when a browser requests http://yoursite/ or http://yoursite/Home, it gets back the output from HomeController's Index method. You can try this yourself by changing the URL in the browser. At the moment, it will be http://localhost:5000/, except that the port part may be different if you are using Visual Studio.
- If you append /Home or /Home/Index to the URL and press Return, you will see the same Hello World result from the application.

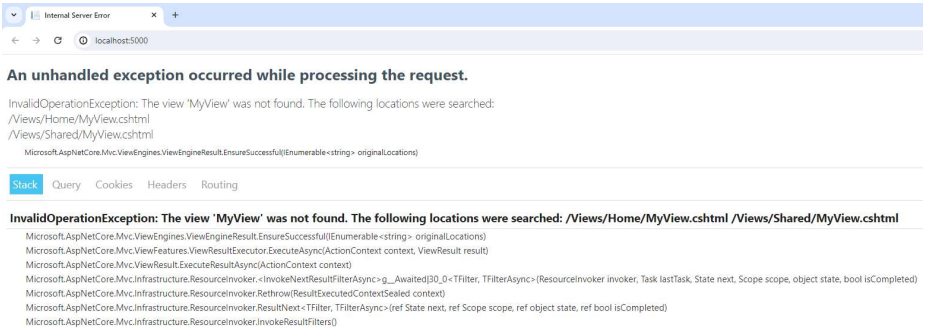
UNDERSTANDING HTML RENDERING

Understanding HTML Rendering

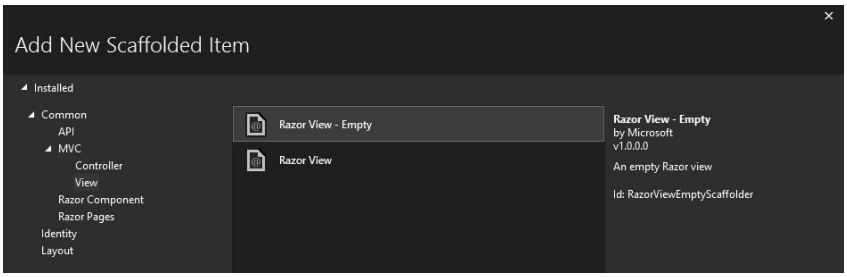
- The output from the previous example wasn't HTML—it was just the string Hello World. To produce an HTML response to a browser request, I need a **view**, which tells ASP.NET Core how to process the result produced by the Index method into an **HTML response** that can be sent to the browser.

Understanding HTML Rendering

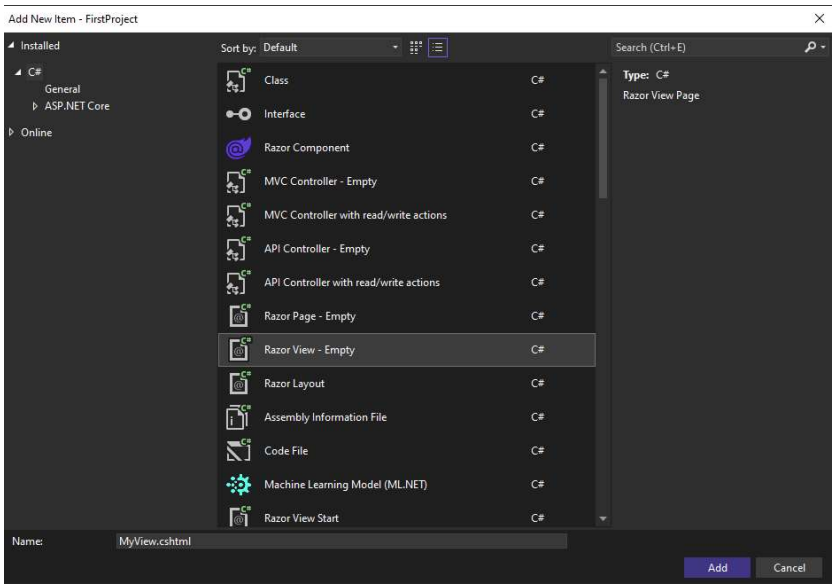
```
0 references
public ViewResult Index()
{
    return View("MyView");
}
```



Understanding HTML Rendering

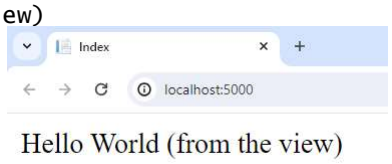


Understanding HTML Rendering



Understanding HTML Rendering

```
@{
 Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width"
/>
    <title>Index</title>
</head>
<body>
    <div>
        Hello World (from the view)
    </div>
</body>
</html>
```



Adding Dynamic Output

- The whole point of a web application is to construct and display dynamic output. The job of the action method is to construct data and pass it to the view so it can be used to create HTML content based on the data values. Action methods provide data to views by passing arguments to the View method
- The data provided to the view is known as the **view model**.

```
0 references
public ViewResult Index()
{
    int hour = DateTime.Now.Hour;
    string viewModel = hour < 12 ? "Good Morning" : "Good Afternoon";
    return View("MyView", viewModel);
}
```

Adding Dynamic Output

```
@model string
@{
 Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Index</title>
</head>
<body>
    <div>
        @Model World (from the view)
    </div>
</body>
</html>
```

Exercise

- Cài đặt và chạy thử chương trình ASP.NET Core theo hướng dẫn đã học
- Tạo một View với tên gọi là LTWeb
- Trong View này load nội dung SV tự chọn, sau đó tạo ActionMethod mới trong HomeController để trả về View này
- Phần đầu trang LTWEB hiển thị thông tin ngày tháng năm hiện tại